T10 Boosting 与梯度提升

让后来的树修正已有预测

机器学习课程学生讲义

辅修版 · 2026 年 11 月 8 日

学生讲义 · 辅修版。对应 T10（2 理论学时），之后完成 B05（2 实验学时）。

导读

T09 用有放回抽样和节点候选特征的随机选择，让多棵树学到有所不同的规则，再汇总它们对同一个对象的预测。在这种训练方式中，一棵树并不需要先知道其他树预测得怎样。本章考虑另一种安排：先得到一个初步预测，再检查它与真实标签还差多少，让后来的树专门学习怎样修正已有预测。

这种逐步组合模型的思路称为 Boosting，通常译为提升；梯度提升是其中一种常见实现。关键不只是增加模型数量，而是后续模型的训练依赖前面已经形成的结果。本章以梯度提升为主线：先用四条回归记录算清一次修正，再用 T04 的梯度概念和 T05 的分类损失说明，为什么分类时也能沿用逐步修正的思路。

> 得到初始预测 → 根据当前损失确定修正方向 → 用一棵浅树学习修正规律 → 按学习率加入已有模型 → 用验证结果判断组合效果。

完成本章后，你应该能手算一次平方损失下的修正，解释学习率与轮数的作用。还应能说明随机森林各树分别训练、梯度提升后续树依赖当前组合结果的区别，比较两者汇总预测的方式，并根据共同验证结果比较固定的模型方案。

1. 后来的树学习什么

1.1 从完整预测转向修正已有预测

假设一个模型对某批次能耗预测为 4，而历史记录中的实际能耗为 7。若希望减少这个误差，预测应当向上调整。另一个批次的预测也是 4，实际却为 1，则应向下调整。已有预测相同，并不意味着各样本需要同方向的修正。

如果输入特征能够区分这些情况，就可以训练一个新的模型，让它根据输入预测需要怎样调整。新模型的输出可以为正，也可以为负：正值把原预测提高，负值把原预测降低。最终预测由已有预

测与新修正相加得到。预测新批次时，只需输入特征就能分别计算这两部分；训练标签用于学习修正规律，并不是预测时额外提供的资料。

1.2 用树表达数值修正

T08 的分类树在叶节点统计类别，输出类别或类别概率。这里需要的修正量是数值，因此使用回归树。它仍然沿特征条件把输入分组，但每个叶节点输出一个数值。在平方损失下，给定落入同一个叶的训练样本的修正目标，用这些目标的均值作为叶输出，可以使这个叶内的平方误差和最小。

例如，两条记录需要的修正分别为 -3 和 -1，若它们进入同一个叶，叶输出可以取均值 -2。同一个叶只能输出一个值，因此不能分别给出 -3 和 -1，只能用同一个值近似这两条记录的修正要求。学习这棵回归树，就是利用输入寻找能把修正要求较相近的记录放在一起的条件，再确定每个叶的输出值。回归树比较划分时看数值目标的平方误差，不使用分类标签的 Gini。

用较浅的树完成每次修正，可以使每一步只表达较简单的规律。这些参与组合的树称为基学习器，这个名称说明它们在组合中的角色，并不限定其预测能力。本章每轮使用表达能力有限的浅树，可借此理解提升方法中“弱学习器”的含义：单棵树能力有限，通过逐步组合形成更有表达能力的模型。

2. 手算一次平方损失下的修正

2.1 初始预测与修正目标

先暂时离开超限分类任务，使用一个抽象回归小表。输入 x 和数值标签 y 如下，只用来观察修正过程。

样本	A	B	C	D
输入 x	1	2	3	4
标签 y	1	3	5	7

把当前组合模型记为 F，$F(x_i)$ 是它对第 i 个样本的数值预测。下标 0 表示尚未加入修正树的初始模型。沿用平方损失下的均值基线，初始常数为：

$$F_0(x) = \frac{1+3+5+7}{4} = 4.$$

T03 把残差统一记为预测减实际，即 $e_i = F_0(x_i) - y_i$。本章把希望补到当前预测上的差额记为 r_i：

$$r_i = y_i - F_0(x_i) = -e_i.$$

因此，四条记录的残差为 $[3, 1, -1, -3]$，修正目标为 $[-3, -1, 1, 3]$。对于 A，当前预测偏高 3，修正目标就是 -3；对于 D，当前预测偏低 3，修正目标就是 3。

2.2 用浅树近似修正目标

给定一棵只分裂一次的回归树，条件为 $x \le 2.5$，把 A、B 放入左叶，把 C、D 放入右叶。这里先使用给定划分，以便集中理解修正过程。左叶的目标均值为 $(-3-1)/2 = -2$，右叶为 $(1+3)/2 = 2$。把这棵修正树记为 h_1，则：

$$h_1(x) = \begin{cases} -2, & x \le 2.5, \\ 2, & x > 2.5. \end{cases}$$

它对四条记录给出的修正是 $[-2, -2, 2, 2]$。这棵树学习的是“在什么输入区域应向上或向下修正”，并没有精确记住每条记录要求的差额。例如，A 希望降低 3，B 希望降低 1，但左叶只能给二者相同的修正值 -2。

2.3 按学习率加入修正

不一定把树给出的修正全部加上。设学习率为 η（读作 eta），本轮更新为：

$$F_1(x) = F_0(x) + \eta h_1(x).$$

取 $\eta = 0.5$，各记录的计算如下。

样本	y_i	$F_0(x_i)$	$r_i = y_i - F_0(x_i)$	$h_1(x_i)$	$\eta h_1(x_i)$	$F_1(x_i)$
A	1	4	-3	-2	-1	3
B	3	4	-1	-2	-1	3
C	5	4	1	2	1	5
D	7	4	3	2	1	5

更新后的预测为 $[3, 3, 5, 5]$。树输出 -2，并不意味着最终预测为 -2；它表示应当对原预测作向下修正，本轮实际加入的量还要乘以学习率。

沿用 T03 的半均方误差：

$$J(F) = \frac{1}{2n} \sum_{i=1}^{n} \big(F(x_i) - y_i\big)^2.$$

这里 $n = 4$。初始残差为 $[3, 1, -1, -3]$，更新后为 $[2, 0, 0, -2]$，所以：

$$J(F_0) = \frac{9 + 1 + 1 + 9}{8} = 2.5, \qquad J(F_1) = \frac{4 + 0 + 0 + 4}{8} = 1.$$

这一步降低了四条训练记录上的损失，但还没有把它们全部拟合正确。是否能帮助新对象，仍需另外的验证记录判断。

3. 为什么称为梯度提升

3.1 修正目标来自损失的下降方向

在平方损失中，“实际减预测”看起来很自然。换一种损失时，应该怎样确定修正方向？T04 用负梯度寻找损失下降的方向；这里也可以沿用同样的依据，但求导的对象是当前模型的输出 F_i，而不是模型参数。

设当前模型对一个样本的输出为 $F_i = F(x_i)$，该样本的损失为：

$$\ell_i = \frac{1}{2}(F_i - y_i)^2.$$

把 y_i 视为已知，对当前输出 F_i 求导，得到：

$$\frac{\partial \ell_i}{\partial F_i} = F_i - y_i, \qquad -\frac{\partial \ell_i}{\partial F_i} = y_i - F_i.$$

因此，第 2 节的修正目标正好是单样本平方损失对当前输出的负梯度。预测偏高时，负梯度为负，要求向下修正；预测偏低时，负梯度为正，要求向上修正。若对整体平均损失 J 求各 F_i 的偏导，还会带有 $1/n$ 因子。本章用各单样本损失的负梯度作为树的学习目标，与前面的数值计算保持一致。

仅知道训练记录应怎样修改，还没有形成面向新输入的规则。新树用输入去拟合这些负梯度，把逐条记录的修正要求近似为一组可以重复使用的条件。树受到深度等限制，因而通常只能近似这些目标；最后还要决定加入多少修正。这就是梯度提升把损失、树的训练和模型组合连接起来的方式。

3.2 每一轮都根据当前组合重新计算

设已经加入了 $m-1$ 棵树，当前模型为 F_{m-1}。先用它计算各训练样本的输出，再根据这些输出重新计算负梯度，训练第 m 棵修正树 h_m。平方损失下的更新写为：

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

对其他可微损失，也可以按这种加法形式逐轮组合模型。修正目标由相应损失的负梯度确定，叶输出的具体计算还需结合所用损失。

第 2 节更新后，新的修正目标已经变成 $y - F_1 = [-2, 0, 0, 2]$。下一棵树应当面对这组新的目标，而不是继续把最初的 $[-3, -1, 1, 3]$ 当作尚未处理的差额。这说明后一棵树的训练为什么必须知道前面模型的结果。

加入 M 棵树以后，模型可以写为：

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x).$$

求和表示把各轮修正累加起来，没有除以树数 M。预测新对象时，每棵已训练的树都根据其输入给出修正，再与初始输出相加。训练顺序有前后依赖，但预测时各棵已训练树的修正可以分别计算，再求和；不需要等到新对象的真实标签出现。

与 T04 相比，两者都利用损失的下降方向；区别在于，T04 直接更新既有线性模型的参数，这里每轮新增一棵表示修正函数的树，再把它加入组合。梯度提升中的“梯度”说明修正方向的来源，不是把树的分裂阈值直接代入线性模型的参数更新式。

3.3 学习率与轮数需要一起考虑

学习率控制每棵修正树加入多少，轮数控制允许加入多少棵。仍用第 2 节已经确定的 h_1，只改变学习率，就会得到不同结果。

学习率 η	更新后的四个预测	训练半均方误差
0.5	$[3, 3, 5, 5]$	1
1	$[2, 2, 6, 6]$	0.5
3	$[-2, -2, 10, 10]$	8.5

在这个固定修正上，$\eta = 1$ 比 $\eta = 0.5$ 使训练损失下降得更多；$\eta = 3$ 却修正过头，损失从初始的 2.5 上升到 8.5。负梯度给出局部下降方向，并不保证沿这个方向走任意大的步子都有效。

较小的学习率使每次改变较少，通常需要更多轮才能形成足够的修正；在固定轮数下，过小的学习率可能使拟合不足。另一方面，继续增加轮数也可能逐步适应噪声和训练记录的特有细节。选择学习率与轮数，应比较预先安排的候选在相同验证条件下的表现，不能仅凭训练损失持续下降决定一直增加树。本次实验固定这些设置，用于观察方法，不扩展参数搜索。

4. 分类时修正分数，而不是给树投票

4.1 从分类概率回到分数

回到能耗超限任务，标签 y 只能取 0 或 1。T05 已区分分数、概率和类别。这里把当前组合输出记为分数 $F(x)$，再通过 Sigmoid 得到超限概率：

$$p = \sigma(F) = \frac{1}{1 + e^{-F}}.$$

同一个符号 F 在第 2 节表示数值回归预测，在本节表示分类分数。分类中的 F 可以为负，也可以大于 1；由它转换得到的 p 才位于 0 与 1 之间。初始分数也应由分类损失确定，不能照搬回归例子中的常数 4。例如，训练中正类比例为 q 且 $0 < q < 1$ 时，让所有对象初始概率都为 q，对应的常数分数可以由 $q = \sigma(F_0) = 1/(1 + e^{-F_0})$ 反解，得到 $F_0 = \ln(q/(1 - q))$；两类各半时，该分数为 0。

对一个样本使用二分类交叉熵：

$$\ell = -\big[y \ln p + (1 - y) \ln(1 - p)\big].$$

利用 T04 的链式法则和 Sigmoid 的导数 $\mathrm{d}p/\mathrm{d}F = p(1 - p)$，可得：

$$\frac{\partial \ell}{\partial F} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p) = p - y.$$

所以交叉熵对分数的负梯度为 $y - p$。例如，实际超限的记录 $y = 1$，当前概率只有 0.2，负梯度为 0.8，要求提高分数，从而提高超限概率；实际正常的记录 $y = 0$，当前概率为 0.7，负梯度为 -0.7，要求降低分数。

负梯度保留了类别判断之外的信息：即使两条超限记录都被判为超限，概率 0.6 与 0.9 对应的负梯度仍分别为 0.4 与 0.1。分类梯度提升不只是挑出判错的记录，也根据损失考虑当前概率离标签还有多远。

4.2 新树提供分数修正

分类梯度提升用这些负梯度寻找输入与修正方向之间的规律，逐轮加入分数修正。这里的加法发生在分数上，不是概率上。组合完成后，先汇总分数，再转换为概率并判断类别。修正树的输出不必是 0 或 1，也不必落在概率范围内；它们承担的是改变组合分数的任务。

第 2 节的平方损失算例可以直接用叶内修正目标均值确定叶输出。分类损失不同，实际训练器还可能在树的分组确定后，按分类损失进一步调整叶输出。因此，$y - p$ 解释的是分类修正的方向与学习目标，不能直接当作所有分类提升实现最终加入的叶值。B05 使用已有的分类梯度提升训练器，观察其组合结果。

把它与 T09 对照：随机森林分别训练各树，再平均各树的类别概率；本章的分类梯度提升逐轮学习分数修正，将修正相加后再通过 Sigmoid 得到概率。树的数量相同，并不意味着两种模型采用相同的训练或汇总方式。

4.3 Boosting 还有其他组织修正的方法

Boosting 是一类逐步组合模型的思路，梯度提升是其中一种具体方法。另一种常见方法 AdaBoost，可从“后续训练更加关注前面难以正确判断的记录”来理解：在常见二分类形式中，它逐轮调整训练记录的权重，使此前错判的记录相对受到更多关注，并把各轮分类器按各自的训练表现加权组合。

调整记录权重，会改变下一轮训练对不同记录的重视程度；拟合负梯度，则为下一棵树提供由当前损失计算出的数值学习目标。两者都依赖已有模型的表现，但不是同一项操作。因此，理解本章时应把“逐步修正”作为共同思路，把“拟合相应损失的负梯度”作为梯度提升的具体机制。

5. 用共同验证比较固定方案

5.1 比较条件包含各自需要的处理

B05 沿用 T07—T09 的批次能耗超限数据：120 条开发记录，输入为计划负荷比例和环境指数，使用共同的分层三折，每折 80 条训练、40 条验证。所有方案在每折训练部分估计中位数填补值；逻辑回归还在这一范围内拟合标准化，数值树模型不增加这一步。

比较使用相同记录、划分与准确率指标，但不要求每种方法采用完全相同的预处理。这里比较的是各自完整的预测流程。预处理需要从数据学习的部分，都只能在当前折的训练范围拟合。四个方案的主要设置预先固定如下。

方案	主要设置
逻辑回归	带正则化的逻辑回归，配合折内标准化
单树	最大深度 5，最小叶样本数 2
随机森林	50 棵树，每棵最大深度 5、最小叶样本数 2，每节点考虑一个候选特征
梯度提升	50 轮，每棵修正树最大深度 1、最小叶样本数 2，学习率 0.1，使用分类交叉熵

这些是固定教学方案，不是各算法经过充分搜索后得到的最佳配置。单树沿用 T09 的深度 5 对照，也不是 T08 验证选择出的深度 1。森林中的树和梯度提升中的浅树复杂度不同，本次可以比较整套方案的结果，不能把差异单独归因于某一个机制。

5.2 分数与错误类型共同描述结果

把每条记录在其验证折得到的预测放回原位置，可得到四组对应相同对象的折外预测。下表来自上述固定设置；漏报是实际超限却预测正常，误报是实际正常却预测超限。

模型	平均训练准确率	平均折内验证准确率	漏报数	误报数
逻辑回归	0.7792	0.7333	16	16
单树	0.9083	0.6333	25	19
随机森林	0.8917	0.6167	24	22
梯度提升	0.8375	0.6583	22	19

单树训练准确率最高，验证准确率却低于逻辑回归。梯度提升在本次比较中比单树少漏报 3 次，误报数相同，因此合计多判对 3 条，平均验证准确率提高 0.025；它仍低于逻辑回归。按本次预定准确率规则，四个预设方案中应选择逻辑回归，不能因为本章介绍了提升方法就预定它胜出。

错误总数的差异，也不等于逐条预测都按同一方向变化。例如，误报数相同，不代表误报的是同一批记录。可以沿用 T09 的成对比较，检查某条记录在哪种方案下被纠正，又在哪种方案下新增错误。若漏报可能使高能耗批次未获预警，误报可能导致不必要的检查，就应分别说明两类错误的后果；实际使用的主指标需要根据任务事先确定，而不是看到结果后改挑有利指标。

这些记录此前已经参与过课程中的开发比较。折外预测保证对应记录未参与负责预测它的那一折流程的拟合，但整批开发资料并未独立于模型选择过程。若据此选定或调整方案，最终表现仍需另留资料检查。本次结果用于理解固定条件下的方法差异，不作算法普遍优劣的结论。

6. 本章小结

梯度提升从已有预测出发，根据当前损失计算修正目标，让一棵新树学习输入与修正之间的关系，再把修正按学习率加入组合。平方损失下，单样本负梯度为实际减预测；分类交叉熵下，对分数的负梯度为 $y - p$。后续树面对的是更新后的组合结果，因此每轮学习目标会随之改变。

学习率决定每步加入多少，轮数决定修正多少步。增加修正能力可以降低训练误差，也可能适应训练数据中的噪声或特有细节。随机森林和梯度提升采用不同的训练与汇总机制，选择哪套方案仍需依靠共同验证和错误分析。B05 将把已有分类方法放在同一评价安排中比较，并把 T08 的项目计划推进到一个可复核的开发内基线。

7. 活动与实验查阅

7.1 用纸笔追踪一次修正

配套纸笔活动的 A、B 部分对应第 2、3 节：先算 $y - F_0$，再算叶内均值、学习率缩放和更新后的损失。C 部分连接分类修正与模型比较，可结合第 4、5 节完成。

7.2 B05 的两项实现

使用 B05 Notebook。四个模型的构造、共同三折、折外预测拼接与绘图都已提供。两个待完成位置分别是 `evaluate(model)` 的验证调用和 `error_counts(actual, pred)` 的混淆计数。先完成两处调用，再用第 5.2 节的实际结果检查汇总是否一致。

在 `evaluate(model)` 内，将完整且尚未拟合的流程 `model`、原始开发输入 `X`、标签 `y` 和共同 `folds` 交给 `cross_validate`。调用时指定 `scoring="accuracy"`，设置 `return_train_score=True`、`return_estimator=True` 和 `error_score="raise"`。输入形状为 `(120, 2)`，标签为 `(120,)`。返回的 `test_score` 是折内验证分数，`train_score` 用于观察拟合情况；`return_estimator=True` 保留每折流程，供已有代码预测该折验证记录。

7.3 把计数对应到同一批记录

`error_counts(actual, pred)` 接收逐行对应的真实标签与类别预测，返回 TP、FP、FN、TN 四项。接口 `confusion_matrix(..., labels=[0, 1]).ravel()` 的顺序为 TN、FP、FN、TP，不能按字典键的书写顺序猜测。例如，真实标签为 `[1, 0, 0]`，预测为 `[0, 1, 0]`，就有一次漏报、一次误报和一次正确的正常判断：`FN=1`、`FP=1`、`TN=1`、`TP=0`。

对每个模型，四种计数的和都应等于所评价的记录数。本次每条开发记录恰好验证一次，合起来为 120 条，且 $(TP + TN)/120$ 应与平均折内验证准确率一致。旧的 40 条验证记录本次不评价。结果表中的 `score_std` 只描述三折分数的波动，不作为置信区间或显著性判断。

逻辑回归调用使用 `LogisticRegression(C=1, max_iter=1000)`。其中 `C` 控制正则化强度，数值越小，正则化越强。本次固定为 1，沿用给定方案进行比较。

Boosting 调用使用 `GradientBoostingClassifier`，主要设置为 `loss="log_loss"`、`n_estimators=50`、`learning_rate=0.1`、`max_depth=1` 和 `min_samples_leaf=2`。这里的 50 表示顺序修正轮数，0.1 缩放各轮的分数修正，不是分类概率阈值。模型调用已提供，重点是用前文机制解释其与森林的区别。

保留程序的检查代码，完成后删除相应的停止提示。查看错误示例时，应按记录编号抽取，不只展示对某个模型有利的记录。在实验 Notebook 中说明实际分数、漏报与误报，并选一个错误解

释可能后果；一条记录只能提供检查线索，不能直接证明错误成因。最后重启内核运行全部，记录辅助工具用途，保存到非保护盘并另留副本。

7.4 将项目计划推进到开发内基线

T08 的持续项目预测批次质量是否不合格，与课堂能耗超限案例是不同任务。按基线接入说明，把基线单元格片段追加到本人原有项目 Notebook，保留此前的问题定义与评价计划。本次只建立一个多数类基线，代码已提供。

多数类基线在当前训练部分统计哪类最多，再对验证对象统一预测这一类。它不使用特征数值，所以这一步不需要填补特征；后续使用特征的候选仍需按训练范围拟合预处理。若多数类恰好是正常，基线会对所有对象预测正常；它可能有不低的准确率，却漏掉所有实际不合格批次。因此应同时查看混淆计数，按原计划的主指标解释局限。

运行前确认 `MODE`、`SEED` 和 `PRIMARY_METRIC` 与原计划一致。提供的模式包括分层三折 `cv` 和开发集内部留出 25% 的固定验证 `holdout`，只运行计划选定的一种。主指标可为准确率、召回率或精确率；若没有任何预测正类，精确率分母为零，程序以 `NaN` 表示未定义，不能把它当作有效成绩。原计划尚未明确时，先记录任务依据再运行。

程序保存 `baseline_protocol`，记录模式、种子、主指标和实际划分索引，供后续候选沿用同一比较安排。项目保留测试资料不在本次使用范围内；这里得到的是开发内基线。解释继续写入同一个项目 Notebook，下一阶段的候选想法可以先记录，本次不另训练复杂模型，也不新增一份重复报告。

8. 自查与迁移

先独立作答，再核对解析。计算可以保留分数。

1. 当前预测为 4、实际标签为 7。按 T03 的约定，残差是多少？平方损失下本章使用的修正目标又是多少？为什么更新时采用加上修正的写法？

2. 对 $x = [1, 2, 3, 4]$、$y = [2, 4, 8, 10]$，初始预测均为标签均值。给定浅树把前两条、后两条各分为一叶。写出修正目标、两叶输出，以及学习率为 0.5 时的更新预测和更新前后的半均方误差。

3. 对第 2 节的小表，学习率为 1 时得到 $[2, 2, 6, 6]$。下一轮修正目标是什么？能否继续把第一轮的 $[-3, -1, 1, 3]$ 当作当前差额？

4. 同一棵修正树用学习率 1 比 0.5 得到更低的训练损失，能否据此规定所有任务都使用 1？增加修正轮数时，应看什么证据？

5. 二分类中，一条记录 $y = 1, p = 0.3$，另一条 $y = 0, p = 0.8$。交叉熵对分数的负梯度分别是多少？它们要求分数往哪个方向改变？为什么不能直接把这些值当作更新后的概率？

6. 随机森林和梯度提升都使用 50 棵树。它们训练后一棵树时，是否都需要前面模型的预测？分类时是否都平均各树的概率？

7. 两个模型的误报都是 19 次，能否说明它们误报的是同一批记录？第 5.2 节中梯度提升比单树多判对几条，能否据此宣称提升方法普遍更好？

8. 项目基线始终预测正常，准确率为 0.8。能否仅凭这一数字认定基线已经满足识别不合格批次的要求？没有预测正类时，精确率应怎样解释？

答案要点与回看位置

1. 残差为 $4-7=-3$；修正目标为 $7-4=3$，也就是残差的相反数。当前预测偏低，正修正加到原预测上使其提高；实际加入多少还要乘以学习率。回看第 2.1、2.3 节。

2. 初始常数为 6，修正目标为 $[-4,-2,2,4]$，两叶分别输出 -3、3。更新预测为 $[4.5,4.5,7.5,7.5]$。初始半均方误差为 $(16+4+4+16)/8=5$；更新后为 $(6.25+0.25+0.25+6.25)/8=13/8$。更新后损失下降，但仍有 $13/8$，说明一轮修正还未完全拟合。回看第 1.2、2.1—2.3 节。

3. 新目标为 $[-1,1,-1,1]$，应由 $y-F_1$ 重新计算。第一轮目标针对的是初始模型，继续把它当作当前差额会忽略已加入的修正。回看第 3.2 节。

4. 不能。这只是给定数据、给定修正下的一步结果。学习率与轮数共同影响拟合，训练改善也可能伴随过拟合，应按预定安排比较验证表现。回看第 3.3 节。

5. 分别为 $y-p=0.7$、$y-p=-0.8$；正负号表示应提高第一条记录的分数、降低第二条记录的分数。它们是相对于当前分数的修正方向，树还需学习修正规律；更新分数后再经 Sigmoid 得到概率。回看第 4.1—4.2 节。

6. 随机森林的各树训练不依赖前面树的组合预测；梯度提升的后续树根据当前组合的损失计算学习目标。本章森林平均各树的正类概率；分类梯度提升累加各轮的分数修正，再经 Sigmoid 转换为概率。回看第 3.2、4.2 节及 T09。

7. 不能确定对象相同，需要逐行对照。梯度提升少漏报 3 次、误报数相同，因此多判对 3 条。这只能描述当前固定设置和共同三折下的结果，不证明算法普遍占优。回看第 5.1—5.2 节。

8. 不能。若验证对象中确实存在不合格批次，始终预测正常会把它们全部漏报，正类召回率为 0。没有预测正类时，精确率的分母为零，未定义；应如实解释，不把它当作高分或据此改挑指标。回看第 7.4 节及 T02。